

Lesson 12: Hearing It Happen

Unity

VIDEO GAME CREATION

This week: your game finally sounds like a game. Four moments - firing, hitting, clearing a wave, and losing - each get their own distinct cue.

Open your own [YourName]Game project from Lesson 11 - don't start a new one.

If you already built an optional sound idea in an earlier lesson (Lessons 2, 3, 4, or 6), keep it - today reuses it directly instead of replacing it. If you didn't, today's steps are written as if starting fresh.

Setup

1. Open your [YourName]Game project from Lesson 11.
2. Import sfx_wave.wav - today's one genuinely new clip - alongside your existing sfx_shoot.wav, sfx_hit.wav, sfx_gameover.wav, sfx_lifelost.wav, and (if you built it) sfx_win.wav.

Level 1 - Minimal (MVP)

Four sounds fire at the correct moment, at least once each: a shot, a hit/explosion, a wave-clear (on any non-final wave), and a game over. No silence, no obviously wrong triggers.

3. In GameManager.cs, add a public AudioClip waveClip field, then drag sfx_wave.wav onto it in the Inspector, right next to the gameOverSound/lifeLostSound fields Lesson 8 already added.
4. Confirm the Player's AudioSource already plays sfx_shoot.wav on fire (from Lesson 3) - add it now if it's missing.
5. Confirm the Enemy prefab's AudioSource already plays sfx_hit.wav on destroy (from Lesson 4) - add it now if it's missing.
6. In RegisterKill()'s nested if (from Lesson 11), add AudioSource.PlayClipAtPoint(instance.waveClip, Camera.main.transform.position) to the wave-advance (else) branch - the same pattern instance.winSound/gameOverSound/lifeLostSound already use.
7. Confirm RegisterEscape()'s lives-reaches-zero branch already calls AudioSource.PlayClipAtPoint(instance.gameOverSound, ...) (from Lesson 8) - add the field and the line now if either is missing. This sound is mandatory starting today even if you skipped it before.
8. Press Play and play through a full round - shoot, hit, an early wave clear, and a loss should each sound distinct.

Highlighted lines are new this week; everything else is code you already have.

```
public static void RegisterKill() {
    enemiesRemaining--;
    score += 10;
    UpdateHUD();
    if (enemiesRemaining <= 0) {
        if (wave >= totalWaves) {
            gameActive = false;
            winText.text = "You Win! Press R to Restart";
            winText.gameObject.SetActive(true);
            AudioSource.PlayClipAtPoint(instance.winSound,
Camera.main.transform.position); // your Lesson 6 Level 3 sound
        } else {
            wave++;
            enemiesRemaining = 5;
            UpdateHUD();
            AudioSource.PlayClipAtPoint(instance.waveClip,
Camera.main.transform.position);
            nextWaveRequested = true;
        }
    }
}

public static void RegisterEscape() {
    lives--;
    UpdateHUD();
    if (lives <= 0) {
        gameActive = false;
        loseText.text = "Game Over - Press R to Restart";
        loseText.gameObject.SetActive(true);
        AudioSource.PlayClipAtPoint(instance.gameOverSound,
Camera.main.transform.position); // your Lesson 8 Level 3 sound, mandatory
as of today
    } else {
        AudioSource.PlayClipAtPoint(instance.lifeLostSound,
Camera.main.transform.position); // your Lesson 8 Level 3 sound
    }
}

// In Update(): if (nextWaveRequested) { nextWaveRequested = false;
spawner.SpawnEnemies(); }
// (RegisterKill() is static and can't reach the instance-only spawner
field directly - same flag trick from Lesson 11.)
```

Alternate Path: Visual Scripting

The same dedicated-per-event pattern works in a Script Graph instead of C#.

9. On the win/wave Branch node from Lesson 11, add an Audio Play Clip One Shot node to each outcome, referencing the winClip/waveClip variables.
10. On the escape graph, confirm an Audio Play Clip One Shot node already references the gameOverClip/lifeLostClip variables (from Lesson 8) in the matching branches - add them now if missing.

Level 2 - Complete (Alpha/Beta)

All four sounds fire reliably every time, with no missing or duplicate playback, and each is clearly distinguishable from the others by ear.

- Play a full round: shoot, hit, an early wave clear, and a loss each produce a distinct sound.
- If you built the optional win sound, confirm it still plays on the true win, and waveClip doesn't also fire on that same frame.
- Confirm waveClip, like winSound/lifeLostSound/gameOverSound, is just one more Inspector-assigned AudioClip reached through instance - no new mechanism needed, same field-and-PlayClipAtPoint pattern as before.

Level 3 - Refined (Ready to Publish) - pick any that interest you

- Audio: if you haven't already, add the optional win sound now - it's the same instance.winSound pattern Lesson 6 built, alongside today's waveClip.
- Audio: add slight pitch variation to hits by setting the Enemy's AudioSource.pitch to a random value (e.g. Random.Range(0.9f, 1.1f)) right before Play().
- Structure: notice every round-ending sound (winSound, waveClip, lifeLostSound, gameOverSound) is just one more Inspector-assigned AudioClip field on GameManager, reached the same way through instance - four simple fields instead of one more complicated shared AudioSource.

If Something's Wrong

- No sound on wave-clear: confirm the waveClip PlayClipAtPoint() call is actually inside the else branch, not after the if/else.
- NullReferenceException on instance.waveClip: confirm the AudioClip was actually dragged onto the Inspector field - the field existing in code isn't enough.
- Game-over or life-lost sound plays repeatedly: confirm the lose branch is still guarded by "if (lives <= 0)" / "else" as a one-time check (from Lesson 8), not re-triggering every frame.

Want More?

Student Resource List > Unity section, or learn.unity.com